Native + Flutter Hybrid App

Development Guide

Step-by-Step Implementation Process

Android (Kotlin) + iOS (Swift) + Flutter Module

Version 1.0

1. Project Overview 21.1 Architecture Summary 21.2 Technology Stack 21.3 Application Flow 32. Phase 1: Flutter Module Development 32.1 Create Flutter Module Project 32.2 Define App Configuration Models 42.3 Implement Native Communication Channel 52.4 Implement Network Layer 62.5 Build Product List Screen 72.6 Build Product Detail Screen 82.7 Module Entry Point 93. Phase 2: Android Native Development 103.1 Project Setup 103.2 Implement Login Activity 113.3 Implement Main Activity with Flutter 124. Phase 3: iOS Native Development 134.1 Project Setup 134.2 Implement Login View 144.3 Implement Flutter Coordinator 155. Phase 4: Integration & Testing 165.1 Integration Testing Checklist 165.2 Build Generation 176. Final Project Structure 187. Development Timeline 198. Deliverables Summary 2

Note: Right-click the Table of Contents and select "Update Field" to refresh page numbers.

1. Project Overview

1.1 Architecture Summary

This project implements a hybrid mobile application architecture where native Android and iOS applications embed a Flutter module for core functionality. The architecture leverages the strengths of both native development (platform-specific features, performance optimization) and cross-platform Flutter development (shared UI components, rapid iteration).

The native layer handles authentication flow, session management, and platform integrations, while the Flutter module manages the product listing, detail views, and payment initiation flow. Communication between layers occurs through MethodChannel on both platforms, enabling bidirectional data exchange.

1.2 Technology Stack

The following table summarizes the core technologies used across all three components of the application:

Component	Technology
Android Native	Kotlin, SDK 35, Jetpack Compose/XML
iOS Native	Swift 5.9+, SwiftUI/UIKit
Flutter Module	Flutter 3.16+, Dart 3.2+
State Management	Riverpod (recommended) / Bloc / Provider
Network Layer	Dio with interceptors for logging
API Source	FakeStoreAPI (https://fakestoreapi.com)

Table 1: Technology Stack Overview

1.3 Application Flow

The application follows a sequential flow that ensures seamless integration between native and Flutter layers while maintaining state consistency across platform boundaries:

Launch App → Check UUID Cache → (If no UUID) Show Login Screen → Generate UUID → Pass Configuration to Flutter → Load Flutter Module → Display Product List → Navigate to Detail → Initiate Payment → Return Payload to Native.

2. Phase 1: Flutter Module Development

2.1 Create Flutter Module Project

Begin by creating a Flutter module (not a full application) that will be embedded within the native apps. A Flutter module differs from a Flutter app in that it is designed to be integrated into existing applications rather than running standalone.

Step 1: Initialize Flutter Module

Execute the following commands in your terminal to create the Flutter module structure:

```
$ flutter create --template=module flutter_product_module
$ cd flutter_product_module
$ flutter pub get
```

Step 2: Configure Dependencies (pubspec.yaml)

Update the pubspec.yaml file with the necessary dependencies:

```
dependencies:
  flutter:
    sdk: flutter
  flutter_riverpod: ^2.4.9
  riverpod_annotation: ^2.3.3
  dio: ^5.4.0
  pretty_dio_logger: ^1.3.1
  cached_network_image: ^3.3.1
  shared_preferences: ^2.2.2
  freezed_annotation: ^2.4.1

dev_dependencies:
  flutter_test:
    sdk: flutter
  build_runner: ^2.4.8
  riverpod_generator: ^2.3.9
  freezed: ^2.4.6
  json_serializable: ^6.7.1
```

2.2 Define App Configuration Models

Create strongly-typed configuration models that will receive data from the native layer. These models define the contract between native and Flutter, ensuring type safety and clear documentation of expected data structures.

Configuration Model (lib/models/app_config.dart)

```
import 'package:freezed_annotation/freezed_annotation.dart';
```

```
part 'app_config.freezed.dart';
part 'app_config.g.dart';

enum ThemeMode { light, dark }

@freezed
class AppConfig with _$AppConfig {
  const factory AppConfig({
    required String uuid,
    required ThemeMode themeMode,
    required ThemeColors colors,
    required SpacingConfig spacing,
    required TypographyConfig typography,
  }) = _AppConfig;

  factory AppConfig.fromJson(Map<String, dynamic> json) =>
      _$AppConfigFromJson(json);
}

@freezed
class ThemeColors with _$ThemeColors {
  const factory ThemeColors({
    required String primary,
    required String secondary,
    required String background,
    required String surface,
    required String textPrimary,
    required String textSecondary,
    required String error,
    required String success,
  }) = _ThemeColors;

  factory ThemeColors.fromJson(Map<String, dynamic> json) =>
      _$ThemeColorsFromJson(json);
}
```

2.3 Implement Native Communication Channel

Establish a MethodChannel-based communication system between Flutter and native platforms. This channel serves as the bidirectional bridge for passing configuration data from native to Flutter and returning payment payloads from Flutter to native.

Platform Channel Service (lib/services/native_channel_service.dart)

```
import 'package:flutter/services.dart';
import 'package:riverpod_annotation/riverpod_annotation.dart';
import '../models/app_config.dart';

part 'native_channel_service.g.dart';

@riverpod
NativeChannelService nativeChannelService(NativeChannelServiceRef ref) {
```

```
  return NativeChannelService();
}

class NativeChannelService {
  static const MethodChannel _channel =
      MethodChannel('com.app.native/flutter');

  Future<AppConfig> getAppConfig() async {
    try {
      final Map<dynamic, dynamic> config =
          await _channel.invokeMethod('getAppConfig');
      return AppConfig.fromJson(Map<String, dynamic>.from(config));
    } on PlatformException catch (e) {
      throw Exception('Failed to get app config: ${e.message}');
    }
  }

  Future<void> initiatePayment(Map<String, dynamic> payload) async {
    try {
      await _channel.invokeMethod('initiatePayment', payload);
    } on PlatformException catch (e) {
      throw Exception('Failed to initiate payment: ${e.message}');
    }
  }
}
```

2.4 Implement Network Layer

Create a robust HTTP client using Dio with automatic request logging and UUID header injection. The network layer abstracts API communication details, providing a clean interface for the data layer.

Dio Client with UUID Interceptor (lib/services/api_client.dart)

```
import 'package:dio/dio.dart';
import 'package:pretty_dio_logger/pretty_dio_logger.dart';

class ApiClient {
  late final Dio _dio;
  String _uuid;

  ApiClient({required String uuid}) : _uuid = uuid {
    _dio = Dio(BaseOptions(
      baseUrl: 'https://fakestoreapi.com',
      connectTimeout: const Duration(seconds: 30),
      receiveTimeout: const Duration(seconds: 30),
      headers: {'Content-Type': 'application/json'},
    ));

    _dio.interceptors.addAll([
      InterceptorsWrapper(
        onRequest: (options, handler) {
          options.headers['X-Session-UUID'] = _uuid;
          return handler.next(options);
```

```
      },
    ),
    PrettyDioLogger(
      requestHeader: true,
      requestBody: true,
      responseBody: true,
      compact: true,
    ),
  ]);
}

Future<List<Product>> getProducts() async {
  final response = await _dio.get('/products');
  return (response.data as List)
      .map((json) => Product.fromJson(json))
      .toList();
}
}
```

2.5 Build Product List Screen

Implement the user interface components that display the product catalog. The list screen uses a grid layout for optimal product visibility.

Product List Screen (lib/screens/product_list_screen.dart)

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import '../providers/product_provider.dart';
import '../providers/config_provider.dart';
import 'product_detail_screen.dart';

class ProductListScreen extends ConsumerWidget {
  const ProductListScreen({super.key});

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final productsAsync = ref.watch(productListProvider);
    final config = ref.watch(appConfigProvider);

    return Scaffold(
      appBar: AppBar(title: const Text('Products')),
      body: productsAsync.when(
        data: (products) => GridView.builder(
          padding: EdgeInsets.all(config.spacing.medium),
          gridDelegate: SliverGridDelegateWithFixedCrossAxisCount(
            crossAxisCount: 2,
            childAspectRatio: 0.7,
          ),
          itemCount: products.length,
          itemBuilder: (context, index) => ProductCard(
            product: products[index],
            onTap: () => Navigator.push(
              context,
```

```
              MaterialPageRoute(
                builder: (_) => ProductDetailScreen(
                  product: products[index],
                ),
              ),
            ),
          ),
        ),
        loading: () => const Center(child: CircularProgressIndicator()),
        error: (err, _) => Center(child: Text('Error: $err')),
      ),
    );
  }
}
```

2.6 Build Product Detail Screen

The detail screen provides comprehensive product information with a clear call-to-action for payment.

Product Detail Screen (lib/screens/product_detail_screen.dart)

```
class ProductDetailScreen extends ConsumerWidget {
  final Product product;
  const ProductDetailScreen({super.key, required this.product});

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final config = ref.watch(appConfigProvider);

    return Scaffold(
      appBar: AppBar(title: Text(product.title)),
      body: SingleChildScrollView(
        padding: EdgeInsets.all(config.spacing.large),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Center(child: Image.network(product.image, height: 250)),
            SizedBox(height: config.spacing.large),
            Text(product.title, style: TextStyle(fontSize: 20)),
            Text('\$${product.price.toStringAsFixed(2)}'),
            Text(product.description),
            SizedBox(height: config.spacing.extraLarge),
            ElevatedButton(
              onPressed: () => _initiatePayment(context, ref),
              child: const Text('Pay Now'),
            ),
          ],
        ),
      ),
    );
  }

  void _initiatePayment(BuildContext context, WidgetRef ref) async {
```

```
    final payload = {
      'productId': product.id,
      'title': product.title,
      'price': product.price,
    };
    await ref.read(nativeChannelServiceProvider).initiatePayment(payload);
  }
}
```

2.7 Module Entry Point

Configure the module's main entry point to initialize the app configuration and provide it to the widget tree.

Main Entry (lib/main.dart)

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'providers/config_provider.dart';
import 'screens/product_list_screen.dart';

class FlutterModuleApp extends StatelessWidget {
  final Map<String, dynamic>? initialConfig;
  const FlutterModuleApp({super.key, this.initialConfig});

  @override
  Widget build(BuildContext context) {
    return ProviderScope(
      overrides: [
        initialConfigProvider.overrideWithValue(initialConfig),
      ],
      child: const MaterialApp(
        debugShowCheckedModeBanner: false,
        home: ProductListScreen(),
      ),
    );
  }
}
```

3. Phase 2: Android Native Development

3.1 Project Setup

Create an Android project configured for embedding a Flutter module using Kotlin, targeting SDK 35.

Step 1: Create Android Project

In Android Studio, create a new project with the following specifications:

- Template: Empty Activity

- Language: Kotlin
- Minimum SDK: API 24 (Android 7.0)
- Compile SDK: 35

Step 2: Configure Gradle (settings.gradle.kts)

```
include(":app")

// Include Flutter module
setBinding(new Binding([gradle: this]))
evaluate(new File(
  settingsDir.parentFile,
  "flutter_product_module/.android/include_flutter.groovy"
))
```

Step 3: Configure App Build (app/build.gradle.kts)

```
plugins {
    id("com.android.application")
    id("org.jetbrains.kotlin.android")
}

android {
    namespace = "com.example.nativeapp"
    compileSdk = 35

    defaultConfig {
        applicationId = "com.example.nativeapp"
        minSdk = 24
        targetSdk = 35
        versionCode = 1
        versionName = "1.0"
    }

    compileOptions {
        sourceCompatibility = JavaVersion.VERSION_17
        targetCompatibility = JavaVersion.VERSION_17
    }
}

dependencies {
    implementation(project(":flutter"))
    implementation("androidx.core:core-ktx:1.12.0")
    implementation("androidx.appcompat:appcompat:1.6.1")
}
```

3.2 Implement Login Activity

Create a login screen with basic validation that generates a UUID upon successful authentication.

LoginActivity.kt

```
package com.example.nativeapp

import android.content.Context
import android.content.Intent
import android.os.Bundle
import android.widget.Button
import android.widget.EditText
import android.widget.Toast
import androidx.appcompat.app.AppCompatActivity
import java.util.UUID

class LoginActivity : AppCompatActivity() {
    private lateinit var etUsername: EditText
    private lateinit var etPassword: EditText
    private lateinit var btnLogin: Button

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_login)

        etUsername = findViewById(R.id.etUsername)
        etPassword = findViewById(R.id.etPassword)
        btnLogin = findViewById(R.id.btnLogin)

        btnLogin.setOnClickListener {
            if (validateInput()) performLogin()
        }
    }

    private fun validateInput(): Boolean {
        if (etUsername.text.isEmpty()) {
            etUsername.error = "Username required"
            return false
        }
        if (etPassword.text.length < 4) {
            etPassword.error = "Password must be 4+ chars"
            return false
        }
        return true
    }

    private fun performLogin() {
        val uuid = UUID.randomUUID().toString()
        getSharedPreferences("AppPrefs", MODE_PRIVATE)
            .edit().putString("uuid", uuid).apply()
        startActivity(Intent(this, MainActivity::class.java))
        finish()
    }
}
```

3.3 Implement Main Activity with Flutter

Create the main activity that hosts the Flutter module and handles bidirectional communication through MethodChannel.

MainActivity.kt

```
package com.example.nativeapp

import android.content.Context
import android.content.Intent
import android.os.Bundle
import android.util.Log
import io.flutter.embedding.android.FlutterActivity
import io.flutter.embedding.engine.FlutterEngine
import io.flutter.embedding.engine.FlutterEngineCache
import io.flutter.plugin.common.MethodChannel

class MainActivity : FlutterActivity() {
    companion object {
        private const val CHANNEL = "com.app.native/flutter"
        private const val ENGINE_ID = "my_flutter_engine"
    }

    private lateinit var methodChannel: MethodChannel

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setupMethodChannel()
    }

    private fun setupMethodChannel() {
        methodChannel = MethodChannel(
            flutterEngine!!.dartExecutor.binaryMessenger,
            CHANNEL
        )

        methodChannel.setMethodCallHandler { call, result ->
            when (call.method) {
                "getAppConfig" -> result.success(buildAppConfig())
                "initiatePayment" -> {
                    val payload = call.arguments as? Map<String, Any>
                    handlePaymentRequest(payload)
                    result.success(null)
                }
                else -> result.notImplemented()
            }
        }
    }

    private fun buildAppConfig(): Map<String, Any> {
        val uuid = getSharedPreferences("AppPrefs", MODE_PRIVATE)
            .getString("uuid", "") ?: ""
        return mapOf(
```

```
            "uuid" to uuid,
            "themeMode" to "light",
            "colors" to mapOf(
                "primary" to "#2196F3",
                "background" to "#FFFFFF"
            ),
            "spacing" to mapOf(
                "small" to 8.0,
                "medium" to 16.0,
                "large" to 24.0
            )
        )
    }

    private fun handlePaymentRequest(payload: Map<String, Any>?) {
        Log.d("MainActivity", "Payment: $payload")
        // Navigate to native payment flow
    }
}
```

4. Phase 3: iOS Native Development

4.1 Project Setup

Create an iOS project using the latest Swift version and configure it for Flutter module integration.

Step 1: Create iOS Project

In Xcode, create a new project with the following configuration:

- Template: App
- Interface: SwiftUI
- Language: Swift

Step 2: Configure Podfile

```
platform :ios, '12.0'

flutter_application_path = '../flutter_product_module'
load File.join(flutter_application_path, '.ios', 'Flutter', 'podhelper.rb')

target 'NativeApp' do
  use_frameworks!
  install_all_flutter_pods(flutter_application_path)
end

post_install do |installer|
  flutter_post_install(installer) if defined?(flutter_post_install)
```

```
end
```

4.2 Implement Login View

Create a SwiftUI login view with validation logic and UUID generation.

LoginView.swift

```
import SwiftUI

struct LoginView: View {
    @State private var username: String = ""
    @State private var password: String = ""
    @State private var errorMessage: String?
    @AppStorage("uuid") private var cachedUuid: String?

    var body: some View {
        VStack(spacing: 24) {
            Image(systemName: "person.circle.fill")
                .font(.system(size: 80))
                .foregroundColor(.blue)

            Text("Welcome").font(.largeTitle).fontWeight(.bold)

            VStack(spacing: 16) {
                TextField("Username", text: $username)
                    .textFieldStyle(RoundedBorderTextFieldStyle())
                SecureField("Password", text: $password)
                    .textFieldStyle(RoundedBorderTextFieldStyle())
            }
            .padding(.horizontal)

            if let error = errorMessage {
                Text(error).foregroundColor(.red).font(.caption)
            }

            Button("Login") { performLogin() }
                .frame(maxWidth: .infinity)
                .padding()
                .background(Color.blue)
                .foregroundColor(.white)
                .cornerRadius(10)
                .padding(.horizontal)
        }
        .padding()
    }

    private func performLogin() {
        guard !username.isEmpty else {
            errorMessage = "Username required"
            return
        }
        guard password.count >= 4 else {
            errorMessage = "Password must be 4+ characters"
```

```
            return
        }
        cachedUuid = UUID().uuidString
    }
}
```

4.3 Implement Flutter Coordinator

Create a coordinator that manages the Flutter engine and handles MethodChannel communication.

FlutterCoordinator.swift

```
import SwiftUI
import Flutter

class FlutterCoordinator: ObservableObject {
    static let shared = FlutterCoordinator()
    var flutterEngine: FlutterEngine?
    var methodChannel: FlutterMethodChannel?

    func initialize() {
        flutterEngine = FlutterEngine(name: "io.flutter")
        flutterEngine?.run()

        methodChannel = FlutterMethodChannel(
            name: "com.app.native/flutter",
            binaryMessenger: flutterEngine!.binaryMessenger
        )

        methodChannel?.setMethodCallHandler { [weak self] call, result in
            switch call.method {
            case "getAppConfig":
                result(self?.buildAppConfig())
            case "initiatePayment":
                if let payload = call.arguments as? [String: Any] {
                    self?.handlePaymentRequest(payload: payload)
                }
                result(nil)
            default:
                result(FlutterMethodNotImplemented)
            }
        }
    }

    private func buildAppConfig() -> [String: Any] {
        let uuid = UserDefaults.standard.string(forKey: "uuid") ?? ""
        return [
            "uuid": uuid,
            "themeMode": "light",
            "colors": ["primary": "#2196F3", "background": "#FFFFFF"],
            "spacing": ["small": 8.0, "medium": 16.0, "large": 24.0]
        ]
    }
```

```
    private func handlePaymentRequest(payload: [String: Any]) {
        NotificationCenter.default.post(
            name: NSNotification.Name("PaymentRequested"),
            object: nil,
            userInfo: payload
        )
    }
}

struct FlutterView: UIViewControllerRepresentable {
    func makeUIViewController(context: Context) -> FlutterViewController {
        let coordinator = FlutterCoordinator.shared
        if coordinator.flutterEngine == nil {
            coordinator.initialize()
        }
        return FlutterViewController(
            engine: coordinator.flutterEngine!,
            nibName: nil,
            bundle: nil
        )
    }

    func updateUIViewController(_ vc: FlutterViewController, context:
Context) {}
}
```

5. Phase 4: Integration & Testing

5.1 Integration Testing Checklist

Before generating release builds, verify that all integration points function correctly:

Test Item	Status
UUID generation and caching on login	☐ Pending
App config passed correctly to Flutter	☐ Pending
Theme mode sync (light/dark)	☐ Pending
API calls include UUID in headers	☐ Pending
Product list loads correctly	☐ Pending
Payment payload returns to native	☐ Pending

Table 2: Integration Testing Checklist

5.2 Build Generation

Generate debug builds for both platforms following these procedures.

Android APK

```
# Build Debug APK
$ cd android_project
$ ./gradlew assembleDebug

# APK location:
# app/build/outputs/apk/debug/app-debug.apk
```

iOS IPA

```
# Build via Xcode:
# 1. Open ios_project.xcworkspace
# 2. Select target device "Any iOS Device"
# 3. Product -> Archive
# 4. Organizer -> Distribute App

# Or via command line:
$ xcodebuild -workspace NativeApp.xcworkspace \
    -scheme NativeApp \
    -configuration Debug \
    -archivePath build/NativeApp.xcarchive \
    archive
```

6. Final Project Structure

The final project structure organizes all components for maintainability and ease of navigation:

```
project-root/
├── flutter_product_module/
│   ├── lib/
│   │   ├── main.dart
│   │   ├── models/
│   │   │   ├── app_config.dart
│   │   │   └── product.dart
│   │   ├── providers/
│   │   │   ├── config_provider.dart
│   │   │   └── product_provider.dart
│   │   ├── screens/
│   │   │   ├── product_list_screen.dart
│   │   │   └── product_detail_screen.dart
│   │   └── services/
│   │       ├── api_client.dart
│   │       └── native_channel_service.dart
│   └── pubspec.yaml
│
├── android_native/
│   ├── app/src/main/java/com/example/nativeapp/
│   │   ├── MainActivity.kt
│   │   └── LoginActivity.kt
│   ├── build.gradle.kts
│   └── settings.gradle.kts
│
```

```
├── ios_native/
│   ├── NativeApp/
│   │   ├── Views/
│   │   │   ├── LoginView.swift
│   │   │   └── FlutterView.swift
│   │   └── Coordinators/
│   │       └── FlutterCoordinator.swift
│   ├── NativeApp.xcodeproj
│   └── Podfile
│
└── README.md
```

7. Development Timeline

The following timeline provides realistic estimates for completing each development phase:

Phase	Duration	Days
Flutter Module Development	3-4 days	3-4
Android Native Development	2-3 days	2-3
iOS Native Development	2-3 days	2-3
Integration & Testing	1-2 days	1-2
Build Generation & Documentation	1 day	1
Total Estimated Time	**9-13 days**	**9-13**

Table 3: Development Timeline Estimate

8. Deliverables Summary

Upon completion, the following deliverables should be prepared and made available:

1. Git Repository: Complete source code with proper .gitignore, comprehensive README, and tagged releases.
2. Android Debug APK: Signed debug APK ready for installation on test devices (API 24-35).
3. iOS Debug IPA: Development-signed IPA for TestFlight distribution or direct installation.
4. Build Links: Upload both APK and IPA to Google Drive or similar cloud storage with public sharing links.
5. Documentation: Technical documentation covering architecture decisions and API integration details.